SYSTEM ARCHITECTURE

Healing Loop Architecture

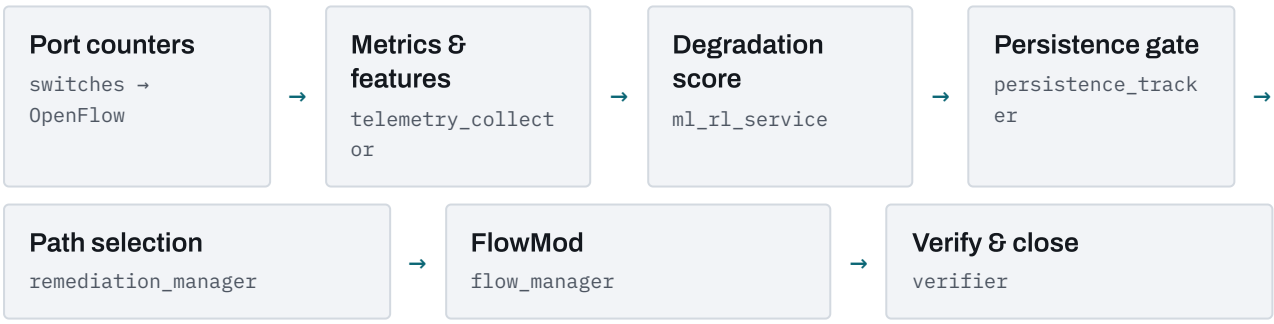
How an AI-driven self-healing software-defined network is put together: the path from a switch counter to an installed flow rule, and the gates in between that decide whether anything happens at all.

Topology 6 switches · 8 links · 5–10 Mbit **Control plane** Ryu · OpenFlow 1.3

Data emulated_mininet

§1 The loop

Switches report port counters. The controller derives per-link metrics from consecutive samples, builds a feature window, and asks a model whether each link is about to degrade. A prediction that *persists* across several windows opens an **incident**. That triggers path selection — a learned policy or a weighted-cost formula — which installs new flow rules. The system then verifies the reroute actually helped and closes the incident as SUCCESS, PARTIAL or FAILED.



BYPASS Hard failures never reach the model. An LLDP link-removal event goes straight from `topology_manager` to `healing_loop.on_link_down`, skipping prediction and the persistence gate entirely — see §6.

Every stage writes to SQLite. That is what makes the experiments and the dashboard possible: each claim traces back to a row.

§2 Where the code runs

Three execution contexts, and the split is forced by dependencies rather than chosen for tidiness. The controller and the model cannot share a

process, so they talk over HTTP.

CONTEXT	PYTHON	HOLDS	WHY SEPARATE
`VM · venv-ctrl`	`3.9`	Ryu controller, `requests`	Ryu pins old dependencies and needs Linux network namespaces
`VM · venv-ml`	`3.11`	Inference service, experiment runner	scikit-learn and torch will not install under 3.9
`macOS · venv-ml`	`3.11`	Training, analysis, figures, dashboard backend	No Mininet needed — iterate without the VM

`controller/of_compat.py` exists so controller modules import cleanly under `venv-ml`, where Ryu is absent, for unit testing — a `TYPE_CHECKING` guard rather than a stubbed Ryu.

§3 Module responsibilities

`common/` Shared vocabulary

`schema.py`

The enums every layer agrees on — LinkHealthState, RouteSelector, DataSource.

`features.py`

Derives metrics from raw counters and builds feature windows. Imported by both the live controller and the offline dataset generator, which is what keeps training and serving on one code path.

`config_loader.py`

Parses config.yaml into frozen dataclasses, so a typo fails at startup rather than mid-experiment.

`controller/` The closed loop

`telemetry_collector.py`

Polls port stats, estimates per-link latency, derives metrics, batches inference calls, and wires the loop together.

`topology_manager.py`

LLDP discovery, the live graph, the flood-safe spanning tree, and link-removal events.

`persistence_tracker.py`

Requires a prediction to repeat before it counts — the gate between "the model twitched" and "act".

`healing_loop.py`

The assembled loop. Deliberately a plain class rather than a RyuApp, so it can be unit-tested with injected sleep, spawn, graph, datapath and metric providers.

`remediation_manager.py`

Chooses a path (learned policy or formula), installs it, and records the decision *and its inputs*.

`path_computer.py`

k-shortest candidates and the weighted-cost ranking used by the formula.

`rl_client.py · inference_client.py`

HTTP to the service, with the safety property that an unreachable service returns nothing rather than raising or guessing.

`verifier.py`

Re-measures after a reroute and decides SUCCESS, PARTIAL or FAILED.

`stability_manager.py`

Cooldown, hold-down and minimum-improvement gates, to stop the loop flapping.

`baselines/`

The comparison systems — B1 static and B2 reactive.

`ml/ · rl/ · service/` The learned parts

`ml/`

Dataset generation, training, threshold sweep, evaluation. Offline only.

`rl/`

The routing policy: a pure-numpy simulator, the Gymnasium environment, state and action encodings, training and evaluation. Also offline — the trained policy is loaded by the service.

`service/ml_rl_service.py`

FastAPI. The only component that loads either model. Exposes degradation probability and path choice, and holds the action mask.

`network/` · `experiments/` The world, and the measurements

`network/failure_injector.py`

Applies real `tc netem` faults with a pre-fault ramp, so the anticipation window has a genuinely worsening signal to learn from.

`experiments/definitions.py`

What each experiment claims — written before it runs.

`experiments/analyze.py`

Aggregates, honestly: marks underpowered cells, separates in-fault detections from reactions to a fault's removal, and drops superseded runs by default.

§4 Data model

Eleven tables, roughly one per stage, so every claim is traceable to a row.
SQLite in WAL mode, stdlib `sqlite3`, no ORM.

TABLE	ONE ROW PER	ANSWERS
`telemetry`	port stats sample	What the switches reported
`features`	derived window, latency probe	What the model was given
`predictions`	scored link	What the model said
`links`	link	Current health state
`topology_snapshots`	topology change	What the graph looked like
`incidents`	incident	What the system decided to fix
`route_changes`	routing decision	What was chosen — **and what it was shown**
`flow_rules`	FlowMod	What was installed
`remediation_events`	step	The audit trail
`verification_results`	verification	Whether it actually helped
`experiment_runs`	trial	The measured outcome

ADDED LATE, FOR A REASON

`route_changes.decision_inputs_json` records the candidates, per-link metrics, degradation probabilities and degraded link at the moment of each decision. Without it, a decision that looks wrong cannot be distinguished from a decision made on different information — and those two call for opposite fixes.

§5 The four systems under test

They exist so a difference between any two is attributable to exactly one mechanism. Each row adds one thing to the row above it, and the same binary runs as any of them via `experiment.system` in config.

SYSTEM	DETECTION	PATH SELECTION
B1 · static	*none*	*none — never reroutes*
B2 · reactive	threshold breach	weighted-cost formula
P1 · ML + heuristic	prediction + persistence	weighted-cost formula
P2 · ML + RL	prediction + persistence	learned policy

B1 is deliberately inert. It is the control that shows what the fault does when nothing responds — and across 29 injected faults of every type, it detected none of them, which is exactly correct behaviour for it.

§6 Two detection paths, not one

Gradual degradation is detected by the model, with persistence required before acting. A hard link failure is **not**. Measured: with a link cut, the model scored it **0.034** — lower than before the cut — because a dead link carries no traffic and returns no probes, and the labels only cover degradation that has yet to happen.

So `topology_manager` emits link-removal events and the healing loop remediates on them directly, bypassing persistence: that gate filters uncertain *predictions*, and a link's disappearance is an observed fact. On the fault type that previously scored 0 of 5, this detects **5 of 5**, recovering in 20.2 s.

WITHHELD FROM B1 ON PURPOSE

Reacting to a link-down event is a control-plane capability, not a learned one. Giving it to the ML arm alone would make P2 beat B2 for a reason that has nothing to do with prediction, so B2, P1 and P2 all get it and B1 stays inert. The two mechanisms are reported separately rather than averaged into one "detection rate", because they work in completely different ways.

§7 Safety properties

The system is built so that failure degrades rather than fabricates.
Experiment E8 tests the first two by killing the inference service mid-run.

An unreachable model means no prediction

Not a guessed one. The client returns nothing and the loop does nothing.

A bad policy answer falls back to the formula

And records the reason for *every* call, not just the failures someone happens to notice.

Every reroute is verified

A failed verification reopens the incident and excludes the failed path, rather than declaring success.

Four gates sit between a prediction and a FlowMod

Persistence, cooldown, hold-down, and minimum improvement.

Unmeasurable is stored as NULL, never as 0

A missing measurement and a measurement of zero are different facts.

§8 What the numbers rest on

Every reported figure traces to a JSON artefact produced by an executed run. Superseded runs are never deleted — they stay in the results database and are listed, with the reason they were discarded, in `invalidated_runs.json`, which the analyser applies by default.

Data is labelled `emulated_mininet` or `synthetic_simulator` at the point of capture. Neither is real-world traffic, and that distinction is carried through to the tables and the figures rather than quietly dropped.

`Design rationale → docs/decisions.md` `Known limitations → docs/limitations.md`